Data Sciences – CentraleSupélec
Advanced Machine Learning
Course III - Linear Classification and Stochastic Optimization

Emilie Chouzenoux

Center for Visual Computing
CentraleSupélec, Inria Saclay
emilie.chouzenoux@centralesupelec.fr

From regression to classification

In regression:

$$y_i \in \mathbb{R}, \qquad f_{\boldsymbol{\beta}}(\mathbf{x}_i) = \mathbf{x}_i^\top \boldsymbol{\beta} \quad \text{predicts a real-valued output.}$$

From regression to classification

In regression:

$$y_i \in \mathbb{R}, \qquad f_\beta(\mathbf{x}_i) = \mathbf{x}_i^\top \beta \quad \text{predicts a real-valued output.}$$

In binary classification:

$$y_i \in \{-1, +1\},$$

and the linear model defines a **score**

$$f_\beta(\mathbf{x}) = \mathbf{x}^\top \beta.$$

As in Course II, $\mathbf{x}$ denotes the augmented feature vector, so that the intercept is included in β.

The predicted class is determined by its sign:

$$\boxed{\widehat{y} = \operatorname{sign}\big(f_\beta(\mathbf{x})\big).}$$

From regression to classification

In regression:

$$y_i \in \mathbb{R}, \qquad f_\beta(\mathbf{x}_i) = \mathbf{x}_i^\top \beta \quad \text{predicts a real-valued output.}$$

In binary classification:

$$y_i \in \{-1, +1\},$$

and the linear model defines a **score**

$$f_\beta(\mathbf{x}) = \mathbf{x}^\top \beta.$$

As in Course II, $\mathbf{x}$ denotes the augmented feature vector, so that the intercept is included in β.

The predicted class is determined by its sign:

$$\boxed{\widehat{y} = \operatorname{sign}\big(f_\beta(\mathbf{x})\big).}$$

The decision boundary is therefore the hyperplane

$$\boxed{\mathbf{x}^\top \beta = 0.}$$

The magnitude $|f_\beta(\mathbf{x})|$ measures the signed separation from the decision boundary.

How should we train a classifier?

For $y_i \in \{-1, +1\}$, define the **margin**

$$\boxed{m_i = y_i f_\beta(\mathbf{x}_i) = y_i \mathbf{x}_i^\top \beta.}$$

How should we train a classifier?

For $y_i \in \{-1, +1\}$, define the **margin**

$$\boxed{m_i = y_i f_{\boldsymbol{\beta}}(\mathbf{x}_i) = y_i \mathbf{x}_i^\top \boldsymbol{\beta}.}$$

The sign of the margin determines whether the prediction is correct:

$$m_i > 0 \quad \Longleftrightarrow \quad \text{correct classification},$$

while $m_i < 0$ corresponds to a misclassified observation.
A large positive margin indicates a confidently classified observation.

How should we train a classifier?

For $y_i \in \{-1, +1\}$, define the **margin**

$$\boxed{m_i = y_i f_\beta(\mathbf{x}_i) = y_i \mathbf{x}_i^\top \beta.}$$

The sign of the margin determines whether the prediction is correct:

$$m_i > 0 \quad \Longleftrightarrow \quad \text{correct classification},$$

while $m_i < 0$ corresponds to a misclassified observation.
A large positive margin indicates a confidently classified observation.
The ideal classification loss is the 0–1 loss:

$$\boxed{\ell_{0/1}(m) = \mathbf{1}_{\{m \leq 0\}}.}$$

However, this loss is

$$\boxed{\text{discontinuous and nonconvex.}}$$

A smooth convex surrogate is the logistic loss; it also has a probabilistic interpretation.

Logistic loss

We replace the 0–1 loss by the smooth convex surrogate

$$\boxed{\ell_{\log}(m) = \log\bigl(1 + \exp(-m)\bigr).}$$

Logistic loss

We replace the 0–1 loss by the smooth convex surrogate

$$\boxed{\ell_{\log}(m) = \log\bigl(1 + \exp(-m)\bigr).}$$

The logistic loss decreases as the margin increases:

$$m \ll 0 \quad \Longrightarrow \quad \text{large loss}, \qquad m \gg 0 \quad \Longrightarrow \quad \text{small loss}.$$

For

$$m_i = y_i f_\beta(\mathbf{x}_i) = y_i \mathbf{x}_i^\top \beta,$$

a large positive margin is therefore strongly favored.

Logistic loss

We replace the 0–1 loss by the smooth convex surrogate

$$\boxed{\ell_{\log}(m) = \log\bigl(1 + \exp(-m)\bigr).}$$

The logistic loss decreases as the margin increases:

$$m \ll 0 \quad \Longrightarrow \quad \text{large loss}, \qquad m \gg 0 \quad \Longrightarrow \quad \text{small loss}.$$

For

$$m_i = y_i f_\beta(\mathbf{x}_i) = y_i \mathbf{x}_i^\top \beta,$$

a large positive margin is therefore strongly favored.

Why this particular surrogate? It also arises naturally from a probabilistic model.

Logistic regression as a Bernoulli model

The sigmoid function

$$\sigma(u) = \frac{1}{1 + \exp(-u)}$$

maps a real-valued score to $(0, 1)$.

Logistic regression as a Bernoulli model

The sigmoid function

$$\sigma(u) = \frac{1}{1 + \exp(-u)}$$

maps a real-valued score to $(0, 1)$.

For binary labels $Y \in \{0, 1\}$, logistic regression assumes

$$\boxed{Y \mid X = \mathbf{x} \sim \text{Bernoulli}(\pi_{\boldsymbol{\beta}}(\mathbf{x})), \qquad \pi_{\boldsymbol{\beta}}(\mathbf{x}) = \sigma(f_{\boldsymbol{\beta}}(\mathbf{x}))\,.}$$

Thus

$$P_{\boldsymbol{\beta}}(Y = 1 \mid X = \mathbf{x}) = \pi_{\boldsymbol{\beta}}(\mathbf{x}), \qquad P_{\boldsymbol{\beta}}(Y = 0 \mid X = \mathbf{x}) = 1 - \pi_{\boldsymbol{\beta}}(\mathbf{x}).$$

Logistic regression as a Bernoulli model

The sigmoid function

$$\sigma(u) = \frac{1}{1 + \exp(-u)}$$

maps a real-valued score to $(0, 1)$.

For binary labels $Y \in \{0, 1\}$, logistic regression assumes

$$\boxed{Y \mid X = \mathbf{x} \sim \text{Bernoulli}(\pi_\beta(\mathbf{x})), \qquad \pi_\beta(\mathbf{x}) = \sigma(f_\beta(\mathbf{x}))\,.}$$

Thus

$$P_\beta(Y = 1 \mid X = \mathbf{x}) = \pi_\beta(\mathbf{x}), \qquad P_\beta(Y = 0 \mid X = \mathbf{x}) = 1 - \pi_\beta(\mathbf{x}).$$

With the equivalent coding $y \in \{-1, +1\}$,

$$\boxed{p_\beta(y \mid \mathbf{x}) = \sigma(y f_\beta(\mathbf{x}))\,.}$$

Hence

$$-\log p_\beta(y_i \mid \mathbf{x}_i) = \log(1 + \exp[-y_i f_\beta(\mathbf{x}_i)]) = \ell_{\log}(y_i f_\beta(\mathbf{x}_i))\,.$$

Logistic loss = Bernoulli negative log-likelihood.

Optimizing logistic regression

Maximum likelihood therefore amounts to minimizing the finite-sum objective

$$\boxed{F(\boldsymbol{\beta}) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(\boldsymbol{\beta}).}$$

with $\ell_i(\boldsymbol{\beta}) = \log\left(1 + \exp(-y_i \mathbf{x}_i^\top \boldsymbol{\beta})\right)$. The gradient of one term is

$$\boxed{\nabla \ell_i(\boldsymbol{\beta}) = -y_i \mathbf{x}_i \, \sigma\left(-y_i \mathbf{x}_i^\top \boldsymbol{\beta}\right).}$$

Hence

$$\boxed{\nabla F(\boldsymbol{\beta}) = \frac{1}{n}\sum_{i=1}^{n} \nabla \ell_i(\boldsymbol{\beta}).}$$

which is Lipschitz continuous with $L = \frac{1}{4n}\|\mathbf{X}\|_2^2$. Gradient-descent iteration, with $0 < \gamma < \frac{2}{L}$, finds a minimizer of F:

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \gamma \nabla F\left(\boldsymbol{\beta}^{(t)}\right).$$

Multiclass classification

Binary logistic regression uses a Bernoulli distribution. For $K > 2$ classes, we use its multiclass analogue: a categorical distribution.

Let $Y_i \in \{1, \ldots, K\}$ denote the class label, represented by its one-hot encoding

$$\mathbf{y}_i \in \{0, 1\}^K, \qquad y_{i,k} = \mathbf{1}_{\{Y_i = k\}}, \qquad \sum_{k=1}^{K} y_{i,k} = 1.$$

Multiclass classification

Binary logistic regression uses a Bernoulli distribution. For $K > 2$ classes, we use its multiclass analogue: a categorical distribution.

Let $Y_i \in \{1, \ldots, K\}$ denote the class label, represented by its one-hot encoding

$$\mathbf{y}_i \in \{0, 1\}^K, \qquad y_{i,k} = \mathbf{1}_{\{Y_i = k\}}, \qquad \sum_{k=1}^{K} y_{i,k} = 1.$$

Define,

$$\boldsymbol{\beta}_k \in \mathbb{R}^{d+1}, \qquad k = 1, \ldots, K, \qquad \mathbf{B} = [\boldsymbol{\beta}_1, \ldots, \boldsymbol{\beta}_K] \in \mathbb{R}^{(d+1) \times K}.$$

For an input $\mathbf{x}_i$, the class scores are $u_{i,k} = \mathbf{x}_i^\top \boldsymbol{\beta}_k$.

Multiclass classification

Binary logistic regression uses a Bernoulli distribution. For $K > 2$ classes, we use its multiclass analogue: a categorical distribution.

Let $Y_i \in \{1, \dots, K\}$ denote the class label, represented by its one-hot encoding

$$\mathbf{y}_i \in \{0,1\}^K, \qquad y_{i,k} = \mathbf{1}_{\{Y_i = k\}}, \qquad \sum_{k=1}^{K} y_{i,k} = 1.$$

Define,

$$\boldsymbol{\beta}_k \in \mathbb{R}^{d+1}, \qquad k = 1, \dots, K, \qquad \mathbf{B} = [\boldsymbol{\beta}_1, \dots, \boldsymbol{\beta}_K] \in \mathbb{R}^{(d+1)\times K}.$$

For an input $\mathbf{x}_i$, the class scores are $u_{i,k} = \mathbf{x}_i^\top \boldsymbol{\beta}_k$.

The **softmax** transforms these scores into probabilities:

$$p_{i,k} = P_{\mathbf{B}}(Y_i = k \mid X_i = \mathbf{x}_i) = \frac{\exp(u_{i,k})}{\sum_{\ell=1}^{K} \exp(u_{i,\ell})}.$$

Thus, conditionally on $\mathbf{x}_i$,

$$\boxed{Y_i \mid X_i = \mathbf{x}_i \sim \text{Categorical}(p_{i,1}, \dots, p_{i,K}).}$$

Multiclass cross-entropy

Hence $p_{i,k} \geq 0$, with $\sum_{k=1}^{K} p_{i,k} = 1$. The predicted class is

$$\boxed{\widehat{y}_i = \arg\max_k p_{i,k} = \arg\max_k u_{i,k}.}$$

For a one-hot encoded label $\mathbf{y}_i$, the negative log-likelihood is

$$\boxed{\ell_i(\mathbf{B}) = -\log p_{\mathbf{B}}(\mathbf{y}_i \mid \mathbf{x}_i) = -\sum_{k=1}^{K} y_{i,k} \log p_{i,k}.}$$

Multiclass cross-entropy

Hence $p_{i,k} \geq 0$, with $\sum_{k=1}^{K} p_{i,k} = 1$. The predicted class is

$$\boxed{\widehat{y}_i = \arg\max_k p_{i,k} = \arg\max_k u_{i,k}.}$$

For a one-hot encoded label $\mathbf{y}_i$, the negative log-likelihood is

$$\boxed{\ell_i(\mathbf{B}) = -\log p_{\mathbf{B}}(\mathbf{y}_i \mid \mathbf{x}_i) = -\sum_{k=1}^{K} y_{i,k} \log p_{i,k}.}$$

Only the probability assigned to the true class contributes to the loss.
This is the **cross-entropy loss**.

Multiclass cross-entropy

Hence $p_{i,k} \geq 0$, with $\sum_{k=1}^{K} p_{i,k} = 1$. The predicted class is

$$\boxed{\widehat{y}_i = \arg\max_k p_{i,k} = \arg\max_k u_{i,k}.}$$

For a one-hot encoded label $\mathbf{y}_i$, the negative log-likelihood is

$$\boxed{\ell_i(\mathbf{B}) = -\log p_{\mathbf{B}}(\mathbf{y}_i \mid \mathbf{x}_i) = -\sum_{k=1}^{K} y_{i,k} \log p_{i,k}.}$$

Only the probability assigned to the true class contributes to the loss.
This is the **cross-entropy loss**.
The empirical objective is

$$\boxed{F(\mathbf{B}) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\mathbf{B}).}$$

The objective is smooth and convex in $\mathbf{B}$, since each loss is a log-sum-exp of linear scores minus a linear term.

Multiclass cross-entropy

Hence $p_{i,k} \geq 0$, with $\sum_{k=1}^{K} p_{i,k} = 1$. The predicted class is

$$\boxed{\widehat{y}_i = \arg\max_k p_{i,k} = \arg\max_k u_{i,k}.}$$

For a one-hot encoded label $\mathbf{y}_i$, the negative log-likelihood is

$$\boxed{\ell_i(\mathbf{B}) = -\log p_{\mathbf{B}}(\mathbf{y}_i \mid \mathbf{x}_i) = -\sum_{k=1}^{K} y_{i,k} \log p_{i,k}.}$$

Only the probability assigned to the true class contributes to the loss.
This is the **cross-entropy loss**.
The empirical objective is

$$\boxed{F(\mathbf{B}) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\mathbf{B}).}$$

The objective is smooth and convex in $\mathbf{B}$, since each loss is a log-sum-exp of linear scores minus a linear term.

Binary logistic regression $\longrightarrow$ multinomial logistic regression.

From full gradient to stochastic optimization

Many machine-learning problems have a **finite-sum structure**

$$F(\beta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\beta).$$

For logistic regression, ℓ_i is the loss associated with observation $(\mathbf{x}_i, y_i)$.

From full gradient to stochastic optimization

Many machine-learning problems have a **finite-sum structure**

$$\boxed{F(\beta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\beta).}$$

For logistic regression, ℓ_i is the loss associated with observation $(\mathbf{x}_i, y_i)$.
Gradient descent uses

$$\nabla F(\beta) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\beta),$$

so that **every iteration processes all n observations**.

From full gradient to stochastic optimization

Many machine-learning problems have a **finite-sum structure**

$$\boxed{F(\beta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\beta).}$$

For logistic regression, ℓ_i is the loss associated with observation $(\mathbf{x}_i, y_i)$.

Gradient descent uses

$$\nabla F(\beta) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\beta),$$

so that **every iteration processes all n observations**.

When n is very large:

$$\boxed{\text{accurate gradient} \quad \Longleftrightarrow \quad \text{expensive iteration.}}$$

From full gradient to stochastic optimization

Many machine-learning problems have a **finite-sum structure**

$$F(\beta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\beta).$$

For logistic regression, ℓ_i is the loss associated with observation $(\mathbf{x}_i, y_i)$.
Gradient descent uses

$$\nabla F(\beta) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\beta),$$

so that **every iteration processes all n observations**.
When n is very large:

accurate gradient $\iff$ expensive iteration.

Key idea: Can we make progress using only a small random subset of the data at each iteration?

cheaper but noisy gradient estimates.

Stochastic Gradient Descent

Consider

$$F(\boldsymbol{\beta}) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\boldsymbol{\beta}).$$

At iteration t, sample an index $i_t \sim \mathcal{U}\{1, \ldots, n\}$, and use the gradient of a single observation:

$$\boxed{\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \gamma_t \nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right).}$$

Stochastic Gradient Descent

Consider

$$F(\boldsymbol{\beta}) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(\boldsymbol{\beta}).$$

At iteration t, sample an index $i_t \sim \mathcal{U}\{1, \ldots, n\}$, and use the gradient of a single observation:

$$\boxed{\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \gamma_t \nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right).}$$

Why is this a reasonable approximation of gradient descent?
Conditionally on the current iterate,

$$\begin{aligned}\mathbb{E}_{i_t}\left[\nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right)\right] &= \frac{1}{n}\sum_{i=1}^{n} \nabla \ell_i\left(\boldsymbol{\beta}^{(t)}\right) \\ &= \nabla F\left(\boldsymbol{\beta}^{(t)}\right).\end{aligned}$$

Hence

$$\boxed{\mathbb{E}_{i_t}\left[\nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right)\right] = \nabla F\left(\boldsymbol{\beta}^{(t)}\right).}$$

Stochastic Gradient Descent

Consider

$$F(\boldsymbol{\beta}) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(\boldsymbol{\beta}).$$

At iteration t, sample an index $i_t \sim \mathcal{U}\{1, \ldots, n\}$, and use the gradient of a single observation:

$$\boxed{\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \gamma_t \nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right).}$$

Why is this a reasonable approximation of gradient descent?
Conditionally on the current iterate,

$$\begin{aligned}\mathbb{E}_{i_t}\left[\nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right)\right] &= \frac{1}{n}\sum_{i=1}^{n} \nabla \ell_i\left(\boldsymbol{\beta}^{(t)}\right) \\ &= \nabla F\left(\boldsymbol{\beta}^{(t)}\right).\end{aligned}$$

Hence

$$\boxed{\mathbb{E}_{i_t}\left[\nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right)\right] = \nabla F\left(\boldsymbol{\beta}^{(t)}\right).}$$

The stochastic gradient is therefore an **unbiased but noisy** estimate of the full gradient.

Stochastic gradient noise

Define the stochastic gradient

$$\mathbf{g}_t = \nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right).$$

We have seen that

$$\mathbb{E}[\mathbf{g}_t \mid \boldsymbol{\beta}^{(t)}] = \nabla F\left(\boldsymbol{\beta}^{(t)}\right).$$

Stochastic gradient noise

Define the stochastic gradient

$$\mathbf{g}_t = \nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right).$$

We have seen that

$$\mathbb{E}[\mathbf{g}_t \mid \boldsymbol{\beta}^{(t)}] = \nabla F\left(\boldsymbol{\beta}^{(t)}\right).$$

However, $\mathbf{g}_t$ fluctuates around the full gradient:

$$\mathbf{g}_t = \nabla F\left(\boldsymbol{\beta}^{(t)}\right) + \boldsymbol{\xi}_t,$$

where

$$\boxed{\mathbb{E}[\boldsymbol{\xi}_t \mid \boldsymbol{\beta}^{(t)}] = 0.}$$

Stochastic gradient noise

Define the stochastic gradient

$$\mathbf{g}_t = \nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right).$$

We have seen that

$$\mathbb{E}[\mathbf{g}_t \mid \boldsymbol{\beta}^{(t)}] = \nabla F\left(\boldsymbol{\beta}^{(t)}\right).$$

However, $\mathbf{g}_t$ fluctuates around the full gradient:

$$\mathbf{g}_t = \nabla F\left(\boldsymbol{\beta}^{(t)}\right) + \boldsymbol{\xi}_t,$$

where

$$\boxed{\mathbb{E}[\boldsymbol{\xi}_t \mid \boldsymbol{\beta}^{(t)}] = 0.}$$

Its variability can be measured by

$$\boxed{\mathbb{E}\left[\|\boldsymbol{\xi}_t\|^2 \,\middle|\, \boldsymbol{\beta}^{(t)}\right].}$$

Large gradient variance means that successive SGD directions may differ substantially.

Stochastic gradient noise

Define the stochastic gradient

$$\mathbf{g}_t = \nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right).$$

We have seen that

$$\mathbb{E}[\mathbf{g}_t \mid \boldsymbol{\beta}^{(t)}] = \nabla F\left(\boldsymbol{\beta}^{(t)}\right).$$

However, $\mathbf{g}_t$ fluctuates around the full gradient:

$$\mathbf{g}_t = \nabla F\left(\boldsymbol{\beta}^{(t)}\right) + \boldsymbol{\xi}_t,$$

where

$$\boxed{\mathbb{E}[\boldsymbol{\xi}_t \mid \boldsymbol{\beta}^{(t)}] = 0.}$$

Its variability can be measured by

$$\boxed{\mathbb{E}\left[\|\boldsymbol{\xi}_t\|^2 \,\middle|\, \boldsymbol{\beta}^{(t)}\right].}$$

Large gradient variance means that successive SGD directions may differ substantially.

SGD trades computational cost per iteration for gradient noise.

Mini-batch Stochastic Gradient Descent

Instead of one observation, sample a mini-batch

$$\mathcal{B}_t \subset \{1, \ldots, n\}, \qquad |\mathcal{B}_t| = b.$$

Define the mini-batch gradient

$$\boxed{\mathbf{g}_t = \frac{1}{b} \sum_{i \in \mathcal{B}_t} \nabla \ell_i\big(\boldsymbol{\beta}^{(t)}\big).}$$

The update becomes

$$\boxed{\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \gamma_t \mathbf{g}_t.}$$

Mini-batch Stochastic Gradient Descent

Instead of one observation, sample a mini-batch

$$\mathcal{B}_t \subset \{1, \dots, n\}, \qquad |\mathcal{B}_t| = b.$$

Define the mini-batch gradient

$$\boxed{\mathbf{g}_t = \frac{1}{b} \sum_{i \in \mathcal{B}_t} \nabla \ell_i\big(\boldsymbol{\beta}^{(t)}\big).}$$

The update becomes

$$\boxed{\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \gamma_t \mathbf{g}_t.}$$

With uniform sampling,

$$\mathbb{E}\left[\mathbf{g}_t \mid \boldsymbol{\beta}^{(t)}\right] = \nabla F\big(\boldsymbol{\beta}^{(t)}\big).$$

The gradient estimate remains unbiased.

Mini-batch Stochastic Gradient Descent

Instead of one observation, sample a mini-batch

$$\mathcal{B}_t \subset \{1, \dots, n\}, \qquad |\mathcal{B}_t| = b.$$

Define the mini-batch gradient

$$\boxed{\mathbf{g}_t = \frac{1}{b} \sum_{i \in \mathcal{B}_t} \nabla \ell_i\big(\boldsymbol{\beta}^{(t)}\big).}$$

The update becomes

$$\boxed{\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \gamma_t \mathbf{g}_t.}$$

With uniform sampling,

$$\mathbb{E}\left[\mathbf{g}_t \mid \boldsymbol{\beta}^{(t)}\right] = \nabla F\big(\boldsymbol{\beta}^{(t)}\big).$$

The gradient estimate remains unbiased.

Increasing the batch size b gives

$$\boxed{\text{less gradient noise} \quad \Longleftrightarrow \quad \text{more computation per iteration.}}$$

Mini-batch Stochastic Gradient Descent

Instead of one observation, sample a mini-batch

$$\mathcal{B}_t \subset \{1, \dots, n\}, \qquad |\mathcal{B}_t| = b.$$

Define the mini-batch gradient

$$\boxed{\mathbf{g}_t = \frac{1}{b} \sum_{i \in \mathcal{B}_t} \nabla \ell_i\big(\boldsymbol{\beta}^{(t)}\big).}$$

The update becomes

$$\boxed{\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \gamma_t \mathbf{g}_t.}$$

With uniform sampling,

$$\mathbb{E}\left[\mathbf{g}_t \mid \boldsymbol{\beta}^{(t)}\right] = \nabla F\big(\boldsymbol{\beta}^{(t)}\big).$$

The gradient estimate remains unbiased.

Increasing the batch size b gives

$$\boxed{\text{less gradient noise} \quad \Longleftrightarrow \quad \text{more computation per iteration.}}$$

Two extreme cases:

$$b = 1 \quad : \quad \text{standard SGD},$$

$$b = n \quad : \quad \text{full gradient descent.}$$

Choosing the learning rate

SGD updates $\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \gamma_t \mathbf{g}_t$. The learning rate $\gamma_t > 0$ controls both **progress** and **sensitivity to gradient noise**.

Choosing the learning rate

SGD updates $\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \gamma_t \mathbf{g}_t$. The learning rate $\gamma_t > 0$ controls both **progress** and **sensitivity to gradient noise**.

Constant learning rate $\gamma_t = \gamma$.

- allows fast initial progress;
- but stochastic fluctuations generally persist near a solution.

Choosing the learning rate

SGD updates $\beta^{(t+1)} = \beta^{(t)} - \gamma_t \mathbf{g}_t$. The learning rate $\gamma_t > 0$ controls both **progress** and **sensitivity to gradient noise**.

Constant learning rate $\gamma_t = \gamma$.

- allows fast initial progress;
- but stochastic fluctuations generally persist near a solution.

Decreasing learning rate

$$\boxed{\gamma_t = \frac{C}{(t + t_0)^{\alpha}}.}$$

Under standard assumptions, classical stochastic-approximation conditions are

$$\boxed{\sum_{t=0}^{\infty} \gamma_t = \infty, \qquad \sum_{t=0}^{\infty} \gamma_t^2 < \infty.}$$

For power-law schedules, this corresponds to $\boxed{\frac{1}{2} < \alpha \leq 1.}$

Choosing the learning rate

SGD updates $\beta^{(t+1)} = \beta^{(t)} - \gamma_t \mathbf{g}_t$. The learning rate $\gamma_t > 0$ controls both **progress** and **sensitivity to gradient noise**.

Constant learning rate $\gamma_t = \gamma$.

- allows fast initial progress;
- but stochastic fluctuations generally persist near a solution.

Decreasing learning rate

$$\boxed{\gamma_t = \frac{C}{(t + t_0)^{\alpha}}.}$$

Under standard assumptions, classical stochastic-approximation conditions are

$$\boxed{\sum_{t=0}^{\infty} \gamma_t = \infty, \qquad \sum_{t=0}^{\infty} \gamma_t^2 < \infty.}$$

For power-law schedules, this corresponds to $\boxed{\frac{1}{2} < \alpha \leq 1.}$

Large steps first, smaller steps as the iterates approach a solution.

SGD with momentum

SGD directions can fluctuate strongly from one iteration to the next.

Momentum accumulates information from previous updates:

$$\boxed{\begin{aligned} \mathbf{v}^{(t+1)} &= \theta\, \mathbf{v}^{(t)} - \gamma_t\, \mathbf{g}_t, \\ \boldsymbol{\beta}^{(t+1)} &= \boldsymbol{\beta}^{(t)} + \mathbf{v}^{(t+1)}, \end{aligned}}$$

with

$$\theta \in [0, 1).$$

SGD with momentum

SGD directions can fluctuate strongly from one iteration to the next.

Momentum accumulates information from previous updates:

$$\boxed{\begin{aligned}\mathbf{v}^{(t+1)} &= \theta\,\mathbf{v}^{(t)} - \gamma_t\,\mathbf{g}_t,\\ \boldsymbol{\beta}^{(t+1)} &= \boldsymbol{\beta}^{(t)} + \mathbf{v}^{(t+1)},\end{aligned}}$$

with

$$\theta \in [0, 1).$$

The velocity $\mathbf{v}^{(t)}$ acts as an exponentially weighted memory of past stochastic gradients.

- directions that persist across iterations are reinforced;
- rapidly oscillating directions tend to compensate;
- progress can be faster along directions of consistent descent.

SGD with momentum

SGD directions can fluctuate strongly from one iteration to the next.

Momentum accumulates information from previous updates:

$$\boxed{\begin{aligned}\mathbf{v}^{(t+1)} &= \theta\,\mathbf{v}^{(t)} - \gamma_t\,\mathbf{g}_t,\\ \boldsymbol{\beta}^{(t+1)} &= \boldsymbol{\beta}^{(t)} + \mathbf{v}^{(t+1)},\end{aligned}}$$

with

$$\theta \in [0, 1).$$

The velocity $\mathbf{v}^{(t)}$ acts as an exponentially weighted memory of past stochastic gradients.

- directions that persist across iterations are reinforced;
- rapidly oscillating directions tend to compensate;
- progress can be faster along directions of consistent descent.

For $\theta = 0$, we recover standard SGD.

Momentum introduces memory into the stochastic update.

Adaptive learning rates: AdaGrad

A single learning rate γ_t treats all coordinates identically.

But different parameters may experience gradients with very different magnitudes.

Adaptive learning rates: AdaGrad

A single learning rate γ_t treats all coordinates identically.
But different parameters may experience gradients with very different magnitudes.
AdaGrad accumulates the squared stochastic gradients:

$$s_j^{(t)} = \sum_{k=1}^{t} \left(g_j^{(k)} \right)^2 .$$

The update is performed coordinatewise:

$$\boxed{\beta_j^{(t+1)} = \beta_j^{(t)} - \frac{\gamma}{\sqrt{s_j^{(t)}} + \varepsilon} g_j^{(t)} .}$$

Adaptive learning rates: AdaGrad

A single learning rate γ_t treats all coordinates identically.
But different parameters may experience gradients with very different magnitudes.
AdaGrad accumulates the squared stochastic gradients:

$$s_j^{(t)} = \sum_{k=1}^{t} \left(g_j^{(k)} \right)^2 .$$

The update is performed coordinatewise:

$$\boxed{\beta_j^{(t+1)} = \beta_j^{(t)} - \frac{\gamma}{\sqrt{s_j^{(t)}} + \varepsilon} g_j^{(t)}.}$$

Thus each coordinate has its own effective learning rate:

$$\boxed{\gamma_{j,t}^{\text{eff}} = \frac{\gamma}{\sqrt{s_j^{(t)}} + \varepsilon}.}$$

- frequently large gradients $\Rightarrow$ smaller future steps;
- rarely updated coordinates retain comparatively larger steps.

Adaptive learning rates: AdaGrad

A single learning rate γ_t treats all coordinates identically.
But different parameters may experience gradients with very different magnitudes.
AdaGrad accumulates the squared stochastic gradients:

$$s_j^{(t)} = \sum_{k=1}^{t} \left(g_j^{(k)} \right)^2 .$$

The update is performed coordinatewise:

$$\boxed{\beta_j^{(t+1)} = \beta_j^{(t)} - \frac{\gamma}{\sqrt{s_j^{(t)}} + \varepsilon} g_j^{(t)} .}$$

Thus each coordinate has its own effective learning rate:

$$\boxed{\gamma_{j,t}^{\text{eff}} = \frac{\gamma}{\sqrt{s_j^{(t)}} + \varepsilon} .}$$

- frequently large gradients $\Rightarrow$ smaller future steps;
- rarely updated coordinates retain comparatively larger steps.

AdaGrad adapts the geometry of the update using past gradients.

From AdaGrad to Adam

In AdaGrad,

$$s_j^{(t)} = \sum_{k=1}^{t} \left(g_j^{(k)} \right)^2 .$$

Since $s_j^{(t)}$ keeps increasing, the effective learning rates may eventually become very small.

From AdaGrad to Adam

In AdaGrad,

$$s_j^{(t)} = \sum_{k=1}^{t} \left(g_j^{(k)} \right)^2 .$$

Since $s_j^{(t)}$ keeps increasing, the effective learning rates may eventually become very small.
RMSProp: replace the cumulative sum by an exponential moving average:

$$\boxed{s_j^{(t)} = \rho\, s_j^{(t-1)} + (1-\rho) \left(g_j^{(t)} \right)^2 .}$$

The update remains

$$\beta_j^{(t+1)} = \beta_j^{(t)} - \frac{\gamma}{\sqrt{s_j^{(t)}} + \varepsilon} g_j^{(t)} .$$

From AdaGrad to Adam

In AdaGrad,

$$s_j^{(t)} = \sum_{k=1}^{t} \left(g_j^{(k)} \right)^2 .$$

Since $s_j^{(t)}$ keeps increasing, the effective learning rates may eventually become very small.
RMSProp: replace the cumulative sum by an exponential moving average:

$$\boxed{s_j^{(t)} = \rho\, s_j^{(t-1)} + (1-\rho) \left(g_j^{(t)} \right)^2 .}$$

The update remains

$$\beta_j^{(t+1)} = \beta_j^{(t)} - \frac{\gamma}{\sqrt{s_j^{(t)}} + \varepsilon} g_j^{(t)}.$$

Adam combines two ideas:

momentum + adaptive coordinate-wise scaling.

It maintains exponential moving averages of both

gradients and squared gradients.

From AdaGrad to Adam

In AdaGrad,

$$s_j^{(t)} = \sum_{k=1}^{t} \left(g_j^{(k)} \right)^2 .$$

Since $s_j^{(t)}$ keeps increasing, the effective learning rates may eventually become very small.
RMSProp: replace the cumulative sum by an exponential moving average:

$$\boxed{s_j^{(t)} = \rho\, s_j^{(t-1)} + (1-\rho) \left(g_j^{(t)} \right)^2 .}$$

The update remains

$$\beta_j^{(t+1)} = \beta_j^{(t)} - \frac{\gamma}{\sqrt{s_j^{(t)}} + \varepsilon} g_j^{(t)}.$$

Adam combines two ideas:

momentum + adaptive coordinate-wise scaling.

It maintains exponential moving averages of both

gradients and squared gradients.

Adam combines gradient memory with adaptive learning rates.

Variance reduction

SGD uses a cheap but noisy estimate $\mathbf{g}_t = \nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right)$ of the full gradient.
Even close to a solution, the individual gradients $\nabla \ell_i(\boldsymbol{\beta})$ do not necessarily vanish.
Thus stochastic gradient noise may persist.

Variance reduction

SGD uses a cheap but noisy estimate $\mathbf{g}_t = \nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right)$ of the full gradient.
Even close to a solution, the individual gradients $\nabla \ell_i(\boldsymbol{\beta})$ do not necessarily vanish.
Thus stochastic gradient noise may persist.

Variance-reduction idea:

Exploit the finite-sum structure

$$F(\boldsymbol{\beta}) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\boldsymbol{\beta})$$

to construct gradient estimates that are

cheap + unbiased or controlled + less noisy.

Variance reduction

SGD uses a cheap but noisy estimate $\mathbf{g}_t = \nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right)$ of the full gradient.
Even close to a solution, the individual gradients $\nabla \ell_i(\boldsymbol{\beta})$ do not necessarily vanish.
Thus stochastic gradient noise may persist.

Variance-reduction idea:

Exploit the finite-sum structure

$$F(\boldsymbol{\beta}) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\boldsymbol{\beta})$$

to construct gradient estimates that are

cheap	+	unbiased or controlled	+	less noisy.

Methods such as

SAG	SAGA	SVRG

combine stochastic updates with additional information from previous or reference gradients.

Variance reduction

SGD uses a cheap but noisy estimate $\mathbf{g}_t = \nabla \ell_{i_t}\left(\boldsymbol{\beta}^{(t)}\right)$ of the full gradient.
Even close to a solution, the individual gradients $\nabla \ell_i(\boldsymbol{\beta})$ do not necessarily vanish.
Thus stochastic gradient noise may persist.

Variance-reduction idea:

Exploit the finite-sum structure

$$F(\boldsymbol{\beta}) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\boldsymbol{\beta})$$

to construct gradient estimates that are

cheap	+	unbiased or controlled	+	less noisy.

Methods such as

SAG	SAGA	SVRG

combine stochastic updates with additional information from previous or reference gradients.

SGD reduces the cost of each iteration;
variance-reduced methods also try to recover the accuracy of full gradient descent.

Stochastic optimization: take-home messages

For large finite-sum problems $F(\beta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\beta)$, the main challenge is to balance

cost per iteration	and	accuracy of the gradient.

Stochastic optimization: take-home messages

For large finite-sum problems $F(\beta) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(\beta)$, the main challenge is to balance

cost per iteration	and	accuracy of the gradient.

- **Full gradient descent:** accurate direction, but processes the entire dataset.
- **SGD:** cheap iterations using unbiased but noisy gradient estimates.
- **Mini-batches:** a compromise between computational cost and gradient variance.
- **Learning-rate schedules and averaging:** control the effect of stochastic noise.
- **Momentum:** introduces memory of previous directions.
- **AdaGrad / RMSProp / Adam:** adapt learning rates using past gradients.
- **Variance-reduced methods:** exploit the finite-sum structure to reduce stochastic noise.

Stochastic optimization: take-home messages

For large finite-sum problems $F(\beta) = \frac{1}{n}\sum_{i=1}^{n} \ell_i(\beta)$, the main challenge is to balance

cost per iteration	and	accuracy of the gradient.

- **Full gradient descent:** accurate direction, but processes the entire dataset.
- **SGD:** cheap iterations using unbiased but noisy gradient estimates.
- **Mini-batches:** a compromise between computational cost and gradient variance.
- **Learning-rate schedules and averaging:** control the effect of stochastic noise.
- **Momentum:** introduces memory of previous directions.
- **AdaGrad / RMSProp / Adam:** adapt learning rates using past gradients.
- **Variance-reduced methods:** exploit the finite-sum structure to reduce stochastic noise.

There is no universally best optimizer:
the appropriate method depends on the structure and scale of the learning problem.